

KNOWLEDGEINSTITUTE OF TECHNOLOGY

(AN AUTONOMOUS INSTITUTION)

Approved by AICTE, New Delhi and Affiliated to Anna University, Chennai

Kakapalayam(Po),Salem-637504



RECORD NOTE BOOK

REG.NO.

Certified that this is the Bonafide record of work done by
Selvan/Selvi of the
IV Semester B.Tech. – Information Technology Branch during the year 2025
-2026 in the BE23CS315- JAVA PROGRAMMING Laboratory.

Staff-Incharge

Head of the Department

Internal Examiner

CONTENTS

EX. NO.	DATE	NAME OF THE EXPERIMENT	PAGE NO.	MARKS	SIGNATURE
1.					
2.					
3.					
4.					
5.					
6.					
7.					
8.					
AVERAGE					

Ex.No.1	Write a program to demonstrate constructor overloading by creating multiple constructors for a class rectangle with different parameters combinations.

AIM

To demonstrate **constructor overloading** by creating multiple constructors for a class Rectangle with different parameter combinations.

PROCEDURE

1. Create a class named Rectangle.
2. Declare variables length and width.
3. Create multiple constructors:
 - Default constructor (no parameters)
 - Constructor with one parameter (square case)
 - Constructor with two parameters (rectangle case)
4. Initialize values inside each constructor.
5. Create a method to calculate area.
6. Create objects using different constructors.
7. Display the results.
8. Compile and execute the program.

PROGRAM

```
class Rectangle {
int length;
int width;
    // Default constructor
Rectangle() {
length = 1;
width = 1;
    }
    // Constructor with one parameter (square)
Rectangle(int side) {
length = side;
width = side;
    }
    // Constructor with two parameters
Rectangle(int l, int w) {
length = l;
width = w;
    }
    // Method to calculate area
int getArea() {
```

```

return length * width;
    }
}
public class ConstructorOverloadingDemo {
public static void main(String[] args) {
    // Using default constructor
    Rectangle r1 = new Rectangle();
    // Using single parameter constructor
    Rectangle r2 = new Rectangle(5);
    // Using two parameter constructor
    Rectangle r3 = new Rectangle(4, 6);
    System.out.println("Area of Rectangle 1: " + r1.getArea());
    System.out.println("Area of Rectangle 2: " + r2.getArea());
    System.out.println("Area of Rectangle 3: " + r3.getArea());
    }
}

```

OUTPUT

Area of Rectangle 1: 1
 Area of Rectangle 2: 25
 Area of Rectangle 3: 24

DEPARTMENT OF CSE		
AIM & ALGORITHM	20	
PROGRAM	30	
EXECUTION & OUTPUT	30	
OBSERVATION	10	
VIVA VOCE	10	
TOTAL	100	

RESULT

Thus, the program to demonstrate constructor overloading using the Rectangle class was executed successfully.

Ex.No.2	Write a program to demonstrate the use of lambda expressions to sort a list of integers in descending order.

AIM

To write a Java program to demonstrate the use of **lambda expressions** to sort a list of integers in **descending order**.

PROCEDURE

1. Import required packages (java.util.*).
2. Create a class and define the main() method.
3. Create a list of integers using ArrayList or Arrays.asList().
4. Display the list before sorting.
5. Use list.sort() method with a **lambda expression** to sort in descending order.
6. Display the list after sorting.
7. Compile and execute the program.

PROGRAM

```
import java.util.*;
public class LambdaSortDescending {
public static void main(String[] args) {
// Creating a list of integers
List<Integer> numbers = new ArrayList<>();
numbers.add(10);
numbers.add(5);
numbers.add(30);
numbers.add(20);
numbers.add(15);
// Display before sorting
System.out.println("Before Sorting: " + numbers);
// Sorting using lambda expression (Descending order)
numbers.sort((a, b) -> b - a);
// Display after sorting
System.out.println("After Sorting (Descending): " + numbers);
}
}
```

OUTPUT

Before Sorting: [10, 5, 30, 20, 15]
After Sorting (Descending): [30, 20, 15, 10, 5]

RESULT

Thus, the Java program to sort a list of integers in descending order using lambda expressions was successfully executed and verified.

DEPARTMENT OF CSE		
AIM & ALGORITHM	20	
PROGRAM	30	
EXECUTION & OUTPUT	30	
OBSERVATION	10	
VIVA VOCE	10	
TOTAL	100	

Ex.No.3	Create a Custom exception InvalidAgeException and use it in a program that checks whether a person's age is within a valid range.

AIM

To create a custom exception **InvalidAgeException** and use it to check whether a person's age is within a valid range.

PROCEDURE

1. Create a custom exception class by extending Exception.
2. Define a constructor to pass an error message.
3. Create a method to validate age.
4. Check whether the age falls within the valid range (e.g., 18 to 60).
5. If invalid, throw the custom exception using throw.
6. Handle the exception using try-catch.
7. Display appropriate output.
8. Compile and execute the program.

PROGRAM

```
class InvalidAgeException extends Exception {
    public InvalidAgeException (String message) {
        super(message);
    }
}

public class AgeValidation {

    // Method to validate age
    static void checkAge(int age) throws InvalidAgeException {
        if (age < 18 || age > 60) {
            throw new InvalidAgeException("Age must be between 18 and 60");
        } else {
            System.out.println("Valid age: " + age);
        }
    }

    public static void main(String[] args) {

        try {
            checkAge(15); // Change value to test
        } catch (InvalidAgeException e) {
            System.out.println("Exception caught: " + e.getMessage());
        }
    }
}
```

OUTPUT

Case 1: Invalid Age

Exception caught: Age must be between 18 and 60

Case 2: Valid Age

Valid age: 25

KNOT

DEPARTMENT OF CSE		
AIM & ALGORITHM	20	
PROGRAM	30	
EXECUTION & OUTPUT	30	
OBSERVATION	10	
VIVA VOCE	10	
TOTAL	100	

RESULT

Thus, the custom exception **InvalidAgeException** was created and used successfully to validate the age range, and the program was executed correctly.

Ex.No.4	Write a program that demonstrates thread synchronization by managing access to a shared resources.

AIM

To demonstrate thread synchronization in Java by controlling concurrent access to a shared resource (Bank Account balance).

PROCEDURE

1. Create a BankAccount class with a shared balance variable.
2. Define a synchronized method for withdrawal to prevent race conditions.
3. Create a Customer class extending Thread.
4. Each thread attempts to withdraw money from the same account.
5. Run multiple threads simultaneously.
6. Observe how synchronization prevents inconsistent balance updates.

PROGRAM

```

class BankAccount {
private int balance = 1000;
    // Synchronized method
public synchronized void withdraw(int amount, String name) {
System.out.println(name + " is trying to withdraw " + amount);
if (balance >= amount) {
System.out.println(name + " is proceeding with withdrawal...");
try {
Thread.sleep(1000); // Simulate delay
} catch (InterruptedException e) {
System.out.println(e);
}
balance -= amount;
System.out.println(name + " completed withdrawal. Remaining balance: " +
balance);
} else {
System.out.println(name + " insufficient balance!");
}
}
}
class Customer extends Thread {
BankAccount account;
int amount;

```



```

Customer(BankAccount account, String name, int amount) {
    super(name);
    this.account = account;
    this.amount = amount;
}
public void run() {
    account.withdraw(amount, getName());
}
}

public class ThreadSyncExample {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();
        Customer c1 = new Customer(account, "Alice", 700);
        Customer c2 = new Customer(account, "Bob", 500);
        c1.start();
        c2.start();
    }
}

```

OUTPUT

Alice is trying to withdraw 700

Alice is proceeding with withdrawal...

Alice completed withdrawal. Remaining balance: 300

Bob is trying to withdraw 500

Bob insufficient balance!

DEPARTMENT OF CSE		
AIM & ALGORITHM	20	
PROGRAM	30	
EXECUTION & OUTPUT	30	
OBSERVATION	10	
VIVA VOCE	10	
TOTAL	100	

RESULT

Thus the thread synchronization was successfully demonstrated. The synchronized method ensures that only one thread accesses the shared resource at a time, preventing race conditions and maintaining data consistency.

Ex.No.5	Write a program that reads a text file and counts the number of words and lines in the file.

AIM

To develop a Java program that reads a text file and counts the number of lines and words present in the file.

PROCEDURE

1. Import required classes from java.io package.
2. Create a BufferedReader object to read the file.
3. Initialize counters for lines and words.
4. Read the file line by line using readLine().
5. Increment line count for each line read.
6. Split each line into words using whitespace delimiter.
7. Count the number of words and add to total.
8. Close the file after reading.
9. Display the total number of lines and words.

PROGRAM

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class FileWordLineCount {
    public static void main(String[] args) {
        int lineCount = 0;
        int wordCount = 0;
        try {
            BufferedReader br = new BufferedReader(new FileReader("input.txt"));
            String line;
            while ((line = br.readLine()) != null) {
                lineCount++;
                String[] words = line.trim().split("\\s+");
                if (!line.trim().isEmpty()) {
```

```

wordCount += words.length;
    }
}

br.close();
System.out.println("Total Lines: " + lineCount);
System.out.println("Total Words: " + wordCount);

    } catch (IOException e) {
System.out.println("Error: " + e.getMessage());
    }
}
}

```

EXAMPLE CONTENT FOR INPUT.TXT

Hello world
This is a Java program

File handling example

OUTPUT

Total Lines: 4
Total Words: 9

DEPARTMENT OF CSE		
AIM & ALGORITHM	20	
PROGRAM	30	
EXECUTION & OUTPUT	30	
OBSERVATION	10	
VIVA VOCE	10	
TOTAL	100	

RESULT

Thus, the Java program to read a text file and count the number of lines and words has been successfully executed and verified.

Ex.No.6	Create a generic method to swap two values of any data type and test it with Integer, String and Double.

AIM

To create a generic method in Java to swap two values of any data type and test it with Integer, String, and Double.

PROCEDURE

1. Create a generic class or method using type parameter <T>.
2. Since Java doesn't support swapping primitive values directly, use a wrapper object (like a class or array).
3. Define a method swap() that exchanges two values.
4. Pass Integer, String, and Double values to test the method.
5. Print values before and after swapping.

PROGRAM

```
class Swapper<T> {
    T value1, value2;
    Swapper(T value1, T value2) {
        this.value1 = value1;
        this.value2 = value2;
    }
    void swap() {
        T temp = value1;
        value1 = value2;
        value2 = temp;
    }
    void display() {
        System.out.println("Value1: " + value1 + ", Value2: " + value2);
    }
}

public class GenericSwapExample {
    public static void main(String[] args) {
        // Integer swap
        Swapper<Integer>intSwap = new Swapper<>(10, 20);
        System.out.println("Before swapping Integers:");
        intSwap.display();
        intSwap.swap();
        System.out.println("After swapping Integers:");
        intSwap.display();
    }
}
```

```

// String swap
Swapper<String>strSwap = new Swapper<>("Hello", "World");
System.out.println("\nBefore swapping Strings:");
strSwap.display();
strSwap.swap();
System.out.println("After swapping Strings:");
strSwap.display();

// Double swap
Swapper<Double>dblSwap = new Swapper<>(1.5, 2.5);
System.out.println("\nBefore swapping Doubles:");
dblSwap.display();
dblSwap.swap();
System.out.println("After swapping Doubles:");
dblSwap.display();
}
}

```

OUTPUT

Before swapping Integers:

Value1: 10, Value2: 20

After swapping Integers:

Value1: 20, Value2: 10

Before swapping Strings:

Value1: Hello, Value2: World

After swapping Strings:

Value1: World, Value2: Hello

Before swapping Doubles:

Value1: 1.5, Value2: 2.5

After swapping Doubles:

Value1: 2.5, Value2: 1.5

DEPARTMENT OF CSE		
AIM & ALGORITHM	20	
PROGRAM	30	
EXECUTION & OUTPUT	30	
OBSERVATION	10	
VIVA VOCE	10	
TOTAL	100	

RESULT

Thus, a generic method was successfully implemented to swap values of different data types such as Integer, String, and Double.

Ex.No.7	Write a program that demonstrates the use of String, String Buffer, and String Builder to manipulate a string, including reversing and appending text.

AIM

To demonstrate the use of String, StringBuffer, and StringBuilder in Java for manipulating strings, including reversing and appending text.

PROCEDURE

1. Create a String object and perform basic operations.
2. Use String Buffer to append text and reverse the string (thread-safe).
3. Use String Builder to append text and reverse the string (faster, not synchronized).
4. Display outputs for each operation.

PROGRAM

```
public class StringDemo {
    public static void main(String[] args) {

        // String (Immutable)
        String str = "Java";
        System.out.println("Original String: " + str);

        String appendedStr = str.concat(" Programming");
        System.out.println("After Appending (String): " + appendedStr);

        String reversedStr = new StringBuilder(str).reverse().toString();
        System.out.println("Reversed (String): " + reversedStr);

        // StringBuffer (Mutable & Thread-safe)
        StringBuffer sbf = new StringBuffer("Java");
        System.out.println("\nOriginalStringBuffer: " + sbf);

        sbf.append(" Programming");
        System.out.println("After Appending (StringBuffer): " + sbf);

        sbf.reverse();
        System.out.println("Reversed (StringBuffer): " + sbf);
    }
}
```

```
// StringBuilder (Mutable & Faster)
StringBuildersbd = new StringBuilder("Java");
System.out.println("\nOriginalStringBuilder: " + sbd);

sbd.append(" Programming");
System.out.println("After Appending (StringBuilder): " + sbd);

sbd.reverse();
System.out.println("Reversed (StringBuilder): " + sbd);
}
```

OUTPUT

Original String: Java
 After Appending (String): Java Programming
 Reversed (String): avaJ

Original StringBuffer: Java
 After Appending (StringBuffer): Java Programming
 Reversed (StringBuffer): gnimmargorPavaJ

Original StringBuilder: Java
 After Appending (StringBuilder): Java Programming
 Reversed (StringBuilder): gnimmargorPavaJ

DEPARTMENT OF CSE		
AIM & ALGORITHM	20	
PROGRAM	30	
EXECUTION & OUTPUT	30	
OBSERVATION	10	
VIVA VOCE	10	
TOTAL	100	

RESULT

Thus, the program successfully demonstrated the usage of String, String Buffer, and String Builder for string manipulation operations like appending and reversing.

Ex.No.8	Write a program to performs CRUD (Create, Read, Update, Delete) operations on a database using JDBC.

AIM

To develop a Java program that performs CRUD (Create, Read, Update, Delete) operations on a database using JDBC.

PROCEDURE

1. Import required JDBC packages.
2. Load and register the JDBC driver.
3. Establish connection to the database.
4. Create a Statement or PreparedStatement.
5. Execute SQL queries for:
 - a. Insert (Create)
 - b. Select (Read)
 - c. Update
 - d. Delete
6. Process the result.
7. Close the connection.

PROGRAM

```
import java.sql.*;

public class JDBC CRUDExample {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/testdb";
        String user = "root";
        String password = "root";
        try {
            // Load driver
            Class.forName("com.mysql.cj.jdbc.Driver");
            // Establish connection
            Connection con = DriverManager.getConnection(url, user, password);
            // CREATE (Insert)
            PreparedStatement psInsert = con.prepareStatement(
                "INSERT INTO student(id, name, age) VALUES (?, ?, ?)");
            psInsert.setInt(1, 1);
            psInsert.setString(2, "Alice");
            psInsert.setInt(3, 20);
            psInsert.executeUpdate();
            System.out.println("Record Inserted");
            // READ (Select)
```



```

        Statement stmt = con.createStatement();
        ResultSets = stmt.executeQuery("SELECT * FROM student");
        System.out.println("\nStudent Records:");
        while (rs.next()) {
            System.out.println(rs.getInt("id") + " " +
                rs.getString("name") + " " +
                rs.getInt("age"));
        }
        // UPDATE
        PreparedStatementpsUpdate = con.prepareStatement(
            "UPDATE student SET age=? WHERE id=?");
        psUpdate.setInt(1, 22);
        psUpdate.setInt(2, 1);
        psUpdate.executeUpdate();
        System.out.println("\nRecord Updated");
        // DELETE
        PreparedStatementpsDelete = con.prepareStatement( "DELETE FROM student
        WHERE id=?");
        psDelete.setInt(1, 1);
        psDelete.executeUpdate();
        System.out.println("Record Deleted");
        con.close();
    } catch (Exception e) {
        System.out.println(e);
    }
}
}

```

OUTPUT

Record Inserted
 Student Records:
 1 Alice 20
 Record Updated
 Record Deleted

DEPARTMENT OF CSE		
AIM & ALGORITHM	20	
PROGRAM	30	
EXECUTION & OUTPUT	30	
OBSERVATION	10	
VIVA VOCE	10	
TOTAL	100	

RESULT

Thus, the Java program to perform CRUD operations using JDBC was successfully executed.